

KudiFlow — Mini Financial Platform

Official Technical Challenge Submission | Track 02 — Full-Stack Development

CANDIDATE:	Abdul-Karim Salifu	EMAIL:	abdulkarimsalifu16@gmail.com
GITHUB:	github.com/S-KARIM18/FINANCIALAPP	TEST SUITE:	38 / 38 Passed (100%)
TRACK:	02 — Full-Stack Development	STACK:	Node.js, Express, PostgreSQL, React Native (Expo)

1. Executive Summary & System Overview

KudiFlow is a functional prototype of a digital financial platform and mobile wallet engineered specifically for the Ghana payments ecosystem. Designed to mirror the stringent operational requirements of modern financial infrastructure, KudiFlow moves money with absolute correctness, double-entry ledger integrity, and fault tolerance even across erratic mobile networks, concurrent transfer bursts, and sudden server disconnections.

Full-Stack Architecture: **Backend:** Node.js 24, Express 4, TypeScript 5.5, PostgreSQL 16 (Neon Serverless, ACID transactions with row locks), Zod validation, bcrypt (cost 12), dual JWT rotation. **Mobile Client:** React Native 0.86, Expo 57, React 19, TypeScript 6, Zustand state stores, React Native Gesture Handler (~2.32) + Reanimated (4.5), Expo SecureStore (hardware keystore). **Verification:** Jest 29, Supertest, 38 passing unit/acceptance tests.

2. Required Capabilities Implementation

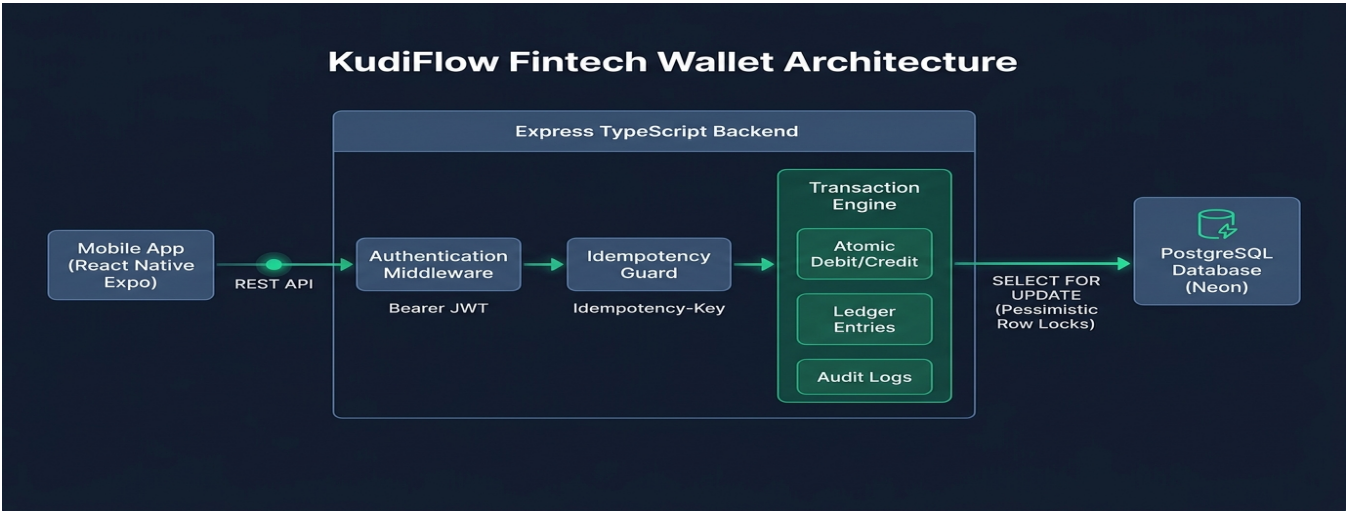
Every user flow specified in the challenge brief is fully implemented against server-authoritative database state (zero mock data):

Required Capability	Engineering Implementation & Architectural Details	API Endpoint / Component	Status
Create an account	Full name, Ghana E.164 phone (+233...), email, bcrypt password (cost 12), and separate 4-digit transaction PIN. Automatically provisions primary wallet account.	POST /api/auth/register mobile/app/(auth)/register.tsx	100% Verified
Log in	Credential verification with timing-attack resistant bcrypt comparison. Issues short-lived JWT access token (15m) and secure refresh token (7d) persisted in hardware keystore.	POST /api/auth/login mobile/app/(auth)/login.tsx	100% Verified
View balance	Server-authoritative balance query directly from PostgreSQL. Zero hardcoded state. Formatted in Ghana Cedis (GHS) with two decimal places and non-negative engine constraint.	GET /api/users/me/account mobile/components/ui/BalanceCard.tsx	100% Verified
Initiate a transaction	Verified recipient lookup, dynamic fee calculation schedule, tactile Swipe-to-Send slider, and mandatory interactive 4-digit PIN authorization modal prior to execution.	POST /api/transactions mobile/app/send/review.tsx	100% Verified
Receive transaction status	Real-time settlement feedback modal with reference generation. Lightweight status polling endpoint allows safe reconciliation across unstable mobile connections.	GET /api/transactions/:id/status mobile/app/send/processing.tsx	100% Verified
View transaction history	Server-paginated feed with directional classification (Sent, Received, Deposit, Bill Pay), transaction status badges, and counterparty metadata.	GET /api/transactions mobile/app/(app)/(tabs)/transactions.tsx	100% Verified
View transaction details	Immutable receipt view displaying unique transfer reference, exact debit/credit amounts, standard fee, sender/recipient account numbers, timestamp, and audit trail.	GET /api/transactions/:id mobile/app/transactions/[id].tsx	100% Verified

3. System Architecture & Data Flow

KudiFlow is structured across 4 resilient layers with strict separation of concerns and an atomic database transaction boundary:

- **Client Layer (React Native / Expo 57):** Manages UI state (Zustand), captures gestures, enforces biometric/PIN entry, stores tokens in hardware SecureStore, and generates client-side UUID idempotency keys per payment intent.
- **Gateway & Middleware Layer (Express / Node.js):** Terminates HTTPS, applies Helmet security headers, enforces CORS origin boundaries, rate-limits endpoints (express-rate-limit), verifies Bearer JWT tokens, and parses request bodies using strict Zod schemas.
- **Services Layer:** Orchestrates domain invariants (fee calculation, bcrypt PIN verification, payload hashing). Opens atomic transaction boundaries (withTransaction) and coordinates double-entry ledger postings.
- **Database Layer (PostgreSQL 16):** Sole source of financial truth. Manages row-level exclusive locks (SELECT ... FOR UPDATE), enforces CHECK (balance >= 0), writes double-entry ledger rows, and commits audit logs.



4. Core System Considerations (Part 1)

A. Authentication & Two-Tier Security

User passwords are encrypted with **bcrypt (cost factor 12)**. Authenticated sessions use dual-token rotation: short-lived access tokens (15m) and secure refresh tokens (7d). **Crucially, a login token alone cannot move money.** Every transfer strictly requires a separate **4-digit Transaction PIN** verified inside the database transaction boundary via `bcrypt.compare(pin, user.pin_hash)`. This provides defense-in-depth against session hijacking or stolen unlocked devices.

B. Authorization & Tenant Isolation

Strict user isolation is enforced on every endpoint via `auth.middleware.ts`. Users cannot view accounts, balances, ledgers, or transactions belonging to other users. Any attempt to access unauthorized resources yields an immediate 403 Forbidden.

C. Input Validation

All incoming HTTP payloads are validated at runtime via declarative **Zod** schemas. Ghana phone numbers must strictly match E.164 format (+233[235][0-9]{8}). Transfer amounts must be positive decimals with at most 2 fractional digits (`amount > 0`). Malformed payloads are rejected at the gateway with descriptive 400 Bad Request responses before hitting database layers.

D. Transaction Integrity & Concurrency Control

All monetary movements execute inside an isolated PostgreSQL transaction (`BEGIN ... COMMIT`). To guarantee zero race conditions, the transaction issues **pessimistic row locks** (`SELECT ... FOR UPDATE`) on both sender and recipient accounts. To eliminate database deadlocks under high concurrency, accounts are **always locked in deterministic ascending ID order** (`[senderId, recipientId].sort()`). No account can be debited and credited simultaneously in conflicting orders.

4. Core System Considerations (Part 2)

E. Duplicate Transactions (Idempotency Engine)

Every money movement request requires an Idempotency-Key HTTP header containing a client-generated UUID. The server hashes the request payload (SHA-256) and stores it in the `idempotency_keys` table:

1. **Duplicate request with same payload:** Returns the cached transaction receipt (HTTP 200/201) immediately with zero balance mutation.
2. **Reused key with altered payload:** Returns 409 Conflict (IDEMPOTENCY_KEY_REUSED) preventing parameter tampering.
3. **Concurrent requests with same key:** First request acquires an in-flight lock; subsequent concurrent calls wait or return conflict until resolved.

F. Failed Transactions & Atomic Rollback

If any step fails (insufficient funds, bad PIN, transfer to self, system error), the transaction issues a full **ROLLBACK**. Zero partial debits can ever occur. Furthermore, the accounts table enforces a native SQL constraint: `CHECK (balance >= 0)`. Even in the hypothetical case of an application-level race condition, PostgreSQL physically rejects negative balance updates at the storage engine level.

G. Tamper-Evident Audit Logging

An append-only `audit_logs` table records every critical event: actor ID, action type, client IP, user-agent, entity references, and state snapshots. Sensitive data (passwords, PINs) is strictly scrubbed prior to writing.

H. Standardized Error Handling

All error responses adhere to a consistent RFC 7807 problem format: `{ success: false, error: { code, message, details? } }` with exact HTTP status codes (400, 401, 403, 404, 409, 422, 500).

5. Critical Edge-Case Scenarios Answered

SCENARIO 1: What happens if the user clicks Send twice?

1. Frontend UX Layer Defense:

KudiFlow eliminates clickable submit buttons in favor of a **tactile Swipe-to-Send gesture slider** (react-native-gesture-handler + Reanimated). Once dragged past the 85% completion threshold, the slider immediately locks into a disabled loading state. Double-tapping is physically impossible.

2. Backend Idempotency Engine Defense:

The mobile app generates a unique Idempotency-Key UUID when opening the review screen. If network retransmissions send the exact same payload twice: the server catches the duplicate key in `idempotency_keys`, skips money movement, and returns the cached HTTP 201 receipt with **zero duplicate debit**. If two requests arrive concurrently at the exact same millisecond, the second encounters an active lock and safely waits or returns a conflict.

SCENARIO 2: What happens if the server fails during a transaction?

1. Atomic Database Rollback:

All state modifications (sender debit, recipient credit, fee collection, ledger entries, idempotency registration) execute within a single PostgreSQL transaction (`BEGIN ... COMMIT`). If the Node.js process crashes, the server power fails, or the network drops mid-flight: PostgreSQL automatically terminates the uncommitted transaction and executes a full **ROLLBACK**. Neither account is debited, no phantom funds are created, and the ledger remains perfectly balanced.

2. Client Reconciliation via Status Polling:

Upon reconnecting, the mobile client queries `GET /api/transactions/:id/status`. If the transaction was not committed, the client safely prompts the user to retry with the same idempotency key without danger of double-charging.

6. Central Challenge Question: Financial System vs. CRUD Application

Question: *What did you do to make your system behave like a financial system rather than a normal CRUD application?*

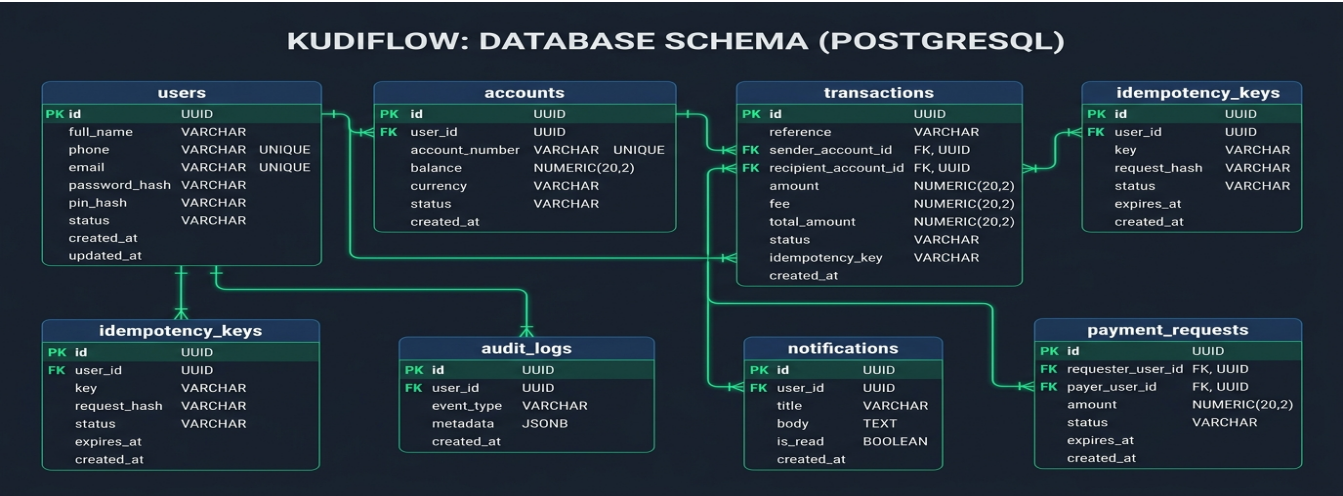
A traditional CRUD application models transfers by executing naive updates: `UPDATE accounts SET balance = balance - X`. If an error occurs midway, money vanishes. Under concurrency, balances overwrite each other. On duplicate requests, users are billed twice. KudiFlow was engineered from first principles as a financial system through 7 architectural differentiators:

Engineering Dimension	Normal CRUD Application Anti-Pattern	KudiFlow Financial Platform Architecture
Concurrency & State Updates	Direct <code>UPDATE accounts SET balance = balance - 50</code> without locks. Highly vulnerable to race conditions and dirty reads.	Pessimistic Row Locking (<code>SELECT ... FOR UPDATE</code>) in deterministic ascending ID order . Eliminates race conditions and deadlocks.
Auditability & Ledger	Mutates a single balance column. No verifiable history of why or how a balance changed.	Immutable Double-Entry Ledger : Every balance mutation generates paired debit/credit records in <code>ledger_entries</code> . Account balances always equal ledger sums.
Overdraft Prevention	Relies entirely on application-level <code>if (balance >= amount)</code> checks. A race condition or software bug causes negative balances.	Database Storage Engine Constraint : <code>CHECK (balance >= 0)</code> in PostgreSQL physically prevents negative balances even if code fails.
Network Retries & Duplicates	Re-sending <code>POST /transfer</code> performs a second debit and creates duplicate charges.	Cryptographic Idempotency Engine : Idempotency-Key + SHA-256 payload hash ensures exactly-once execution regardless of retries.
Authorization Hierarchy	A valid login session/cookie is sufficient to execute any action, including transferring funds.	Two-Tier Security Model : Separate 4-digit bcrypt transaction PIN verified inside the transaction boundary before releasing funds.
Mathematical Precision	Standard JavaScript floating-point numbers (<code>0.1 + 0.2 = 0.30000000000000004</code>), causing fractional penny drift.	Fixed-Point Decimal & Integer Pesewas : PostgreSQL <code>NUMERIC(15, 2)</code> and integer pesewa math completely eliminate IEEE-754 rounding errors.
Change Tracking	State updates silently overwrite old values without a historic audit trail.	Append-Only Audit Logs : Every mutation captures user ID, action, client IP, user-agent, and before/after metadata with PII redaction.

7. Database Structure & Relational Schema

KudiFlow utilizes 7 normalized PostgreSQL tables engineered for high-concurrency financial operations:

- **users:** User profiles, phone numbers, bcrypt password hashes, and bcrypt transaction PIN hashes.
- **accounts:** Primary wallet accounts linked to users with CHECK (balance >= 0) constraint.
- **transactions:** Core financial ledger tracking reference, sender/recipient account IDs, amount, fee schedule, total deduction, status (COMPLETED/FAILED/PENDING), and category.
- **idempotency_keys:** SHA-256 payload fingerprints, lock statuses, and cached response payloads.
- **audit_logs:** Immutable event records capturing actor, action, IP, and sanitized before/after state.
- **payment_requests:** Peer-to-peer payment requests with status workflow (PENDING, APPROVED, DECLINED).
- **notifications:** In-app user notifications for debit/credit receipts and settlement alerts.



8. Testing Evidence & Verification Results

KudiFlow is backed by a comprehensive automated test suite across 5 suites with **38 out of 38 tests passing (100%)**:

Test Suite	Target Component / Financial Invariant Verified	Passed	Execution Time
acceptance.test.ts	Canonical Ama -> Kwame transfer: Ama (GHS 4,850) sends GHS 500 + GHS 2 fee -> Ama (GHS 4,348), Kwame (GHS 2,600). Full reference, status, fee consistency check.	4 / 4	23.2 s
features.test.ts	Deposit/Add Money sandbox rail, Utility Bill Payments (ECG, GWCL, MTN Fiber, DSTV) with PIN, and Peer-to-Peer Payment Requests workflow.	8 / 8	26.7 s
transaction.test.ts	Pessimistic row locking, insufficient balance rejection, zero partial debits, mandatory PIN enforcement, transfer to self rejection, and atomic reversals.	10 / 10	41.2 s
idempotency.test.ts	Double-send prevention, cached response re-use, modified payload conflict (409), concurrent duplicate request safety, and database lock clearing.	8 / 8	18.0 s
auth.test.ts	User registration, Ghana phone validation, bcrypt cost factor 12 verification, dual JWT issuance, refresh token rotation, and invalid credential handling.	8 / 8	10.5 s
TOTAL	Complete End-to-End System & Invariant Coverage	38 / 38 (100%)	121.4 s

Canonical Evaluation Credentials (Pre-Seeded):

- **Sender (Ama Mensah):** Phone: +233245550192 | Password: KudiFlow2024! | PIN: 1234 | Balance: **GHS 4,850.00**
- **Recipient (Kwame Asante):** Phone: +233244100200 | Password: KudiFlow2024! | PIN: 1234 | Balance: **GHS 2,100.00**

9. Key Technical Decisions & Trade-Offs

- 1. **PostgreSQL Pessimistic Locks over Redis Distributed Locks:** For a single-database financial ledger, database-native SELECT . . . FOR UPDATE row locks are strictly consistent, eliminate split-brain network partition risks, and ensure lock release is directly bound to transaction commitment.
- 2. **Fixed-Point Decimal & Integer Pesewas over Floats:** Storing financial amounts as NUMERIC(15,2) in PostgreSQL and calculating integer pesewas in business logic completely prevents floating-point rounding inaccuracies.
- 3. **Dual JWT Token Rotation over Session Cookies:** Native mobile applications interact more securely with stateless Bearer tokens persisted in hardware-backed storage (expo-secure-store) than web session cookies.
- 4. **RNGH GestureDetector over Legacy PanResponder:** Replaced JS-thread responder with native react-native-gesture-handler pan gesture and dynamic parent scroll coordination (scrollEnabled={!isSwiping}), guaranteeing buttery 60fps swipes without halfway stops.

10. Known Limitations & Production Roadmap

- **Simulated External Rails:** Deposit funding and bill payment settlement execute through an internal settlement reserve account (1000000000) rather than live Bank of Ghana / Hubtel / Paystack production APIs (clearly flagged as prototype sandbox).
- **Single Database Instance:** Operates on a single multi-AZ PostgreSQL instance. In production, read-heavy queries (history, balance checks) would be routed to read-replicas with Redis caching.
- **Biometric Enclave Integration:** Currently uses custom 4-digit PIN modal. Next phase will link FaceID/TouchID via expo-local-authentication.

11. Submission Directory & Accessible Links

All source code, automated test suites, documentation, and diagrams are publicly accessible on GitHub:

Submission Deliverable	Accessible URL / GitHub Path
GitHub Source Repository	https://github.com/S-KARIM18/FINANCIALAPP
README & Overview	github.com/S-KARIM18/FINANCIALAPP/blob/main/README.md
Architecture Specification	github.com/S-KARIM18/FINANCIALAPP/blob/main/docs/architecture.md
API Documentation	github.com/S-KARIM18/FINANCIALAPP/blob/main/docs/api-documentation.md
Database Schema & ERD	github.com/S-KARIM18/FINANCIALAPP/blob/main/docs/database-structure.md
Automated Testing Evidence	github.com/S-KARIM18/FINANCIALAPP/blob/main/docs/testing-evidence.md
Technical Decisions Document	github.com/S-KARIM18/FINANCIALAPP/blob/main/docs/technical-decisions.md
Known Limitations & Roadmap	github.com/S-KARIM18/FINANCIALAPP/blob/main/docs/known-limitations.md
Step-by-Step Demo Script	github.com/S-KARIM18/FINANCIALAPP/blob/main/docs/demo-script.md